RAI: Safe Payments Through Committee Changes Without Stopping

Anonymous Author(s)
Regular paper

***Abstract*—Nano and FastPay confirm each account's payments separately, with no global order, in one or two rounds of votes. Two things remain hard. When the validators change, old votes can still complete a confirmation after the handover, so a careless handover can confirm two conflicting payments. And when an account signs two conflicting payments, by mistake or on purpose, most systems leave it stuck. RAI adds epochs. Each epoch ends in a checkpoint built from signed reports of the outgoing validators, and the next epoch votes while the previous one closes, under a few rules that keep the two from contradicting each other. Checkpoints also let a forked account's owner recover it unless a faulty validator backed both payments. We prove safety and termination and model-check the handover rules. On a prototype, RAI keeps confirming through transitions that stall a stop-and-drain handover, with latency tails up to four times shorter.**

***Index Terms*—Byzantine fault tolerance, reconfiguration, asset transfer, consensusless payments, model checking**

I. Introduction

A payment moves money from one account to another. If every account has a single owner, there is no need to put all payments of the world in one sequence: the owner orders the payments of their own account, and the validators only have to make sure that no two incompatible versions of that account's history both become final. Guerraoui et al. showed that this makes asset transfer strictly easier than consensus [1], and systems such as FastPay [2], Astro [3], Zef [4], and Nano [5] use it to confirm a payment in one or two rounds of votes, with no leader and no block interval.

Such systems leave two problems open, and the two are connected.

Changing the validators. Validators join, leave, and move weight. In Nano the voting weight of a validator is whatever the accounts delegating to it hold right now, read by every node from its own ledger, so two honest nodes can count the same vote with different weight and can disagree on who is in the quorum. FastPay fixes the set of validators for the lifetime of the system and lists reconfiguration as future work, because there is "no natural point" at which all authorities switch [2]. The textbook remedy is to stop: end voting, wait until every pending confirmation has settled, agree on a snapshot, and start the new validators from it. Every transition then pauses the system for as long as the slowest confirmation takes.

Accounts that contradict themselves. An owner who signs two conflicting payments for the same position of their account, by a wallet bug or on purpose, splits the validators' votes. If neither side reaches the quorum, no payment at that position can ever be confirmed. In FastPay such an account "may become inaccessible" [2]; in Nano a fork whose final votes split stays unresolved, and every later payment of that account is blocked behind it (we measure this in Section X). A double spend should cost the attacker, not the honest owner who made a mistake, and it should not make the rest of the system slower: the owner should be able to recover the account once they stop contradicting themselves.

RAI. This paper presents RAI, a set of rules that gives an account-local payment system epochs, checkpoints, and committee changes without stopping, and that lets the owner of a forked account recover it unless a faulty validator backed both sides. Time is cut into epochs. In each epoch a fixed committee of validators votes on payments exactly as before. At the end of an epoch every outgoing validator freezes one signed *report* of what it saw; the outgoing committee agrees on one *checkpoint* built from these reports by a fixed recipe; and the next committee, known two epochs ahead, installs the checkpoint without voting on it. The checkpoint settles what the old epoch could still have confirmed and keeps at most one side of every unresolved fork, with a note of what may still happen to it. From then on at most one side can become final, and the owner recovers the account by building on the side the checkpoint kept.

The hard part is the overlap. RAI does not stop at an epoch boundary: validators start voting in epoch $e + 1$ while epoch e is still closing. Votes already given in epoch e can still complete a confirmation after the boundary, and a checkpoint that ignored such a *latent* confirmation could let the next epoch confirm a conflicting payment. Four rules keep the overlap safe (Section VII): new-epoch confirmations wait for the checkpoint unless they carry proof that the old epoch cannot contradict them; outgoing validators keep adding that proof for a short while; votes cast before their signer knows the checkpoint do not count toward the fast path; and a validator does not switch sides across the boundary. Three earlier versions of these rules looked right on paper and were wrong; TLA$^+$ model checking found each failure in a few steps and verifies the repaired rules exhaustively for six validators (Section IX).

Contributions. (1) A protocol for pipelined epoch transitions in account-local payment systems: immutable signed reports, a deterministic checkpoint recipe with *locks* that record what may still become final and which epoch they protect, lagged committees, and the four overlap rules. (2) A proof sketch of safety and termination relative to two abstract services,

set reconstruction and agreement, whose interfaces RAI fixes but whose mechanics it leaves open; full proofs are in the supplement. (3) Model checking of the handover rules, which exposed three failures of earlier rules and checks the repaired rules, including the rule that guards a whole account prefix. (4) A prototype in the RsNano implementation of Nano, evaluated against unmodified Nano and against a stop-and-drain transition on nine fork and fault workloads, with repetitions, with and without emulated link delay, and under validator restarts, committee changes, and a crashing departing validator.

RAI is for authenticated committees with an externally maintained fault bound; it does not address permissionless membership, and its liveness promise to an owner who keeps contradicting themselves is only that their account cannot hurt anyone else.

II. The Problem, by Example

Six validators $V_1, \ldots, V_6$ confirm payments; at most one of them, say V_6, is faulty. A block is confirmed when four validators have given it their *final vote*. Alice signs block A at position 7 of her account, paying Bob, and publishes it.

What a committee change must preserve. Suppose the committee is about to change, and the validators agree on a snapshot of all accounts to hand over. Validators V_1 to V_4 have each signed a final vote for A, but the network is slow: no single party has seen all four votes, and in particular the validators building the snapshot have seen at most three. If the snapshot leaves position 7 open, the new committee may confirm a conflicting block A' there. Later the four old votes meet, and A is confirmed too, by a perfectly valid certificate of the old committee. Two conflicting payments are final. Nothing in the agreement on the snapshot went wrong; the snapshot was simply built from a view of the votes that was not the only possible one. We call A *latent-final*: its certificate exists in the released signatures, though nobody has assembled it. The same holds for a certificate that outgoing validators who have not yet stopped may still complete after the snapshot.

The textbook remedy is to stop: end voting, wait until every certificate has either been assembled or become impossible, then snapshot. The wait is as long as the slowest confirmation, at every transition, and during it no payment is confirmed. Figure 4 in Section X shows what this looks like: confirmations stop for seconds at every boundary.

RAI's answer is that the snapshot does not need to know whether A is final. It needs to know whether A *could* be final, and that is visible in the reports: a correct validator final-votes only a block it has seen notarized, and says so in its report. Four final voters and five reporters out of six overlap in at least three validators, at least two of them correct, so the snapshot sees A notarized in at least two reports and keeps it. The snapshot does not confirm A (it cannot know), but it *locks* position 7 for A, so the new committee cannot confirm A', and A's latent certificate stays compatible with everything that follows.

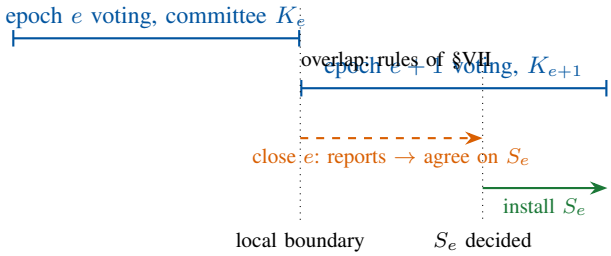


Fig. 1. A pipelined transition. Epoch $e+1$ voting starts at each validator's local boundary while epoch e closes. Most new blocks finalize during the overlap under the overlap rule; the rest finalize when S_e is installed.

What a committee change should not cost. Nano has no epochs; a representative's weight changes whenever a delegating account moves, read by each node from its own ledger. Two correct nodes can thus see different quorums. Stopping is not an option there either, which is one reason the problem has stayed open. RAI wants both: committee changes with a clear, agreed membership and weight per epoch, and no stop.

Accounts that contradict themselves. Now suppose Alice signs two blocks at position 7, A paying Bob and A' paying Carol, and the validators split three against three. No block reaches four. In Nano the position is stuck for good, and so is every later block of Alice's. A deliberate attacker loses access to their own money, which is fine; an honest user with a buggy wallet loses it too, which is not. In RAI the next checkpoint keeps at most one of the two blocks, under a lock, and drops the other, which can then never become final unless a faulty validator also backed it. What RAI guarantees from there is that the owner can recover: once Alice stops contradicting herself and signs a single new block on top of the kept one, that block becomes final, and the kept payment with it. Often she does not even need to: the validators vote the kept block again in the next epoch, and it becomes final on its own. But that is a convenience, not a promise. A deliberate attacker gets at most one of their two payments through. The next sections make these statements precise.

III. RAI at a Glance

Figure 1 shows one epoch transition. Three committees are involved: K_e votes on payments in epoch e and, at the end, decides the checkpoint S_e; K_{e+1} is already known (it was fixed by S_{e-1}) and votes in epoch $e+1$ while K_e is still closing; S_e fixes K_{e+2}. Boundaries are local: each validator ends epoch e on its own clock, freezes its report, and opens $e+1$ at once. Nothing requires the validators to stop together.

RAI is a set of rules. We state them here informally, with the thresholds of the six-validator example in parentheses, and spend the next sections explaining why each is needed.

Inside an epoch (Section V).

R1 A validator casts at most one *first vote* per account position per epoch, for the first valid block it sees there, and never switches.

R2 A block with q (4) first votes is *notarized*. A validator casts a *final vote* only for the block it first-voted, once it holds that block's notarization. A block is *final* with q (4) final votes, or on the fast path with $N - p$ (5) first votes from validators that know the previous checkpoint.

At the boundary (Section VI).

R3 At its local boundary a validator stops voting in epoch e, signs one immutable *report* naming what it holds final or notarized and what else it first-voted, and opens epoch $e + 1$.

R4 The outgoing committee agrees on a checkpoint built from $N - f$ (5) reports by a fixed recipe: keep what any report shows final; *lock* a block that a report shows notarized; *lock for recovery* a block that r (3) reporters first-voted; promote a notarized block that is the only one kept at its position; carry every older lock forward unless evidence of its own origin epoch releases it.

R5 The next committee installs the decided checkpoint without voting on it. A rival of a locked block is refused while the lock stands. A block kept only for recovery is built on only once it is notarized again; the owner recovers the account by extending it.

While the previous epoch closes (Section VII).

R6 *Gate.* A block of epoch $e+1$ does not become final before S_e is installed, except under R7.

R7 *Overlap rule.* It may become final early if it, and every unresolved block below it, has q (4) supporters in epoch e and q (4) first votes in epoch $e + 1$, and if every older recovery lock it bypasses is released by a witness of that lock's own origin.

R8 *Late notarization.* A validator that has left epoch e but not yet installed S_e also signs an epoch-e *support* for each block it first-votes in $e + 1$, unless it supported a rival at that position in e. These supports count only toward the q of R7, never toward finality or reports.

R9 *Early votes.* A first vote cast before its signer installed S_e counts toward notarization and toward R7, never toward the fast path. An early final vote is released only with the witnesses of R7 in hand.

R10 *No switching across the boundary.* Until S_e is installed, a validator does not first-vote in $e + 1$ a block that conflicts with one it supported at that position in e.

Two services are left abstract: how a validator obtains the sets behind another's report (*set reconstruction*) and how the outgoing committee picks one eligible checkpoint (*agreement*). RAI only states what it needs from them (Section IV).

IV. Model and Assumptions

Participants. Clients own accounts and sign blocks. A committee has $N = 3f + 2p + 1$ validators of equal weight, of which at most f are Byzantine; p is the number of responses the fast path may do without, not an extra fault allowance. Corruption is static: a Byzantine identity stays Byzantine, a correct one stays correct, including after it leaves. Who may join a committee, and that no committee exceeds f faults, are deployment assumptions. Our running example and prototype use $f = p = 1$, $N = 6$; Table I lists the thresholds that follow.

TABLE I
VOTE THRESHOLDS, AND THEIR VALUES FOR $f = p = 1$, $N = 6$.

What	Needs	Example
Block notarized (also: exclusion witness)	$q = N - f - p$ first votes or supports	4
Block final, normal path	q final votes	4
Block final, fast path	$N - p$ settled first votes	5
Recovery lock in a checkpoint	$r = f + p + 1$ reporter first votes	3
Reports in a checkpoint	$N - f$	5

The numbers are chosen so that any two groups of q validators share at least $f + 1$ members, hence at least one correct one ($2q - N = f + 1$); so that any q final voters and any $N - f$ reporters share at least $p + 1$ correct ones ($q - 2f = p + 1$); and so that any $N - p$ fast voters and any $N - f$ reporters share at least r correct ones ($N - p - 2f = r$). These three overlaps carry every safety argument of the paper.

Accounts and blocks. Each account is a chain of owner-signed blocks starting at a genesis. A block names its parent and sits at the parent's position plus one; positions are heights in the account, not times. Two blocks of one account *conflict* if neither is an ancestor of the other. A block carries payments: a *send* debits its own account; a *receive* credits the account and names a send that must already be final. An owner *equivocates* by signing conflicting blocks. Safety covers equivocating owners; liveness promises them nothing while they keep contradicting themselves.

Network and cryptography. Signatures cannot be forged and hashes do not collide. Every vote binds its epoch, committee, account, position, kind, block, and the evidence under which it was cast, so a vote cannot be replayed in another context. Correct validators write what they sign to disk before sending it, keep blocks, votes, and evidence, and keep serving them after they leave an epoch. Safety needs no bound on message delay. Termination needs the usual: finite work, fair retry, and recurring windows in which messages arrive within a known delay and clocks drift by a bounded factor.

What a checkpoint contains. A checkpoint S_e is the agreed state at the end of epoch e: for each account the finalized prefix, the unresolved blocks the epoch kept, and *locks* on those blocks; plus balances and the identifiers of received sends. A lock says "keep this block, it may still be final" and records its *strength* (notarized, or recovery) and its *origin*: the epoch whose possible confirmation it protects. A lock is released only by evidence from its own origin epoch (Section VI). The committee of epoch $e + 2$ is a deterministic function of S_e, so it is known throughout the transition from $e + 1$ to $e + 2$.

Two abstract services. RAI commits to sets by their hash and to candidates by their digest, and uses two services to

turn these commitments into shared state. *Set reconstruction* lets any validator obtain the two sets a reporter committed to in its signed report; RAI needs that a reconstructed set matches the signed hash (integrity) and that a correct reporter's sets are eventually reconstructible at every correct validator (termination). *Agreement* lets the outgoing committee decide one candidate checkpoint among those RAI declares *eligible*; RAI needs that all correct validators decide the same value, that the decided value is eligible, that the decision proof is verifiable by anyone who knows the committee and cannot be replayed, and that a decision is reached whenever $N - f$ correct members hold an eligible candidate. Any set-reconciliation mechanism and any Byzantine agreement protocol with external validity [6] fit; our prototype uses rateless set reconciliation [7] and a Kudzu-style election [8]. The supplement states the interfaces formally (C1–C2, A1–A4).

V. Inside an Epoch

Within an epoch RAI votes like Nano or FastPay, with two twists that pay off at the boundary: a validator never switches sides, and a final vote is preceded by a notarization.

Confirming a block. Alice signs block A at position 7 of her account and sends it to the six validators. Each validator checks the block against Alice's chain as it knows it (parent, balance, no duplicate receive) and, if it has not voted at position 7 in this epoch, casts a first vote for A (R1). With four first votes A is *notarized*; a validator that holds the notarization and first-voted A casts its final vote. Four final votes make A final. If five first votes arrive from validators that have installed the previous checkpoint, A is final at once, in one round trip; this is the common case, and the reason RAI's median latency in Section X is below Nano's, which always needs a second round.

Why the thresholds are safe. Any two groups of four validators out of six share at least two, and at most one of those is Byzantine. A correct validator supports one block per position (R1), so two conflicting blocks cannot both reach four supporters. Four supporters for A therefore *exclude* every rival: no rival can be notarized or final in this epoch. We call a set of q supporters an *exclusion witness*; it is the one piece of evidence the boundary rules will ask for again and again. Five first votes for A leave at most one correct validator free to support a rival, so a fast-final block has no rival with three honest supporters either; this is what makes the recovery threshold $r = 3$ of R4 work.

A double spend. Suppose Alice also signs A' at position 7, paying someone else, and shows A to three validators and A' to the other three (Figure 2). Neither reaches four. In Nano this position is stuck: final votes cannot be withdrawn, no block at position 7 will ever confirm, and every later block of Alice's account waits behind it. In RAI the next checkpoint decides what may still happen at position 7 (R4). It is built from five of the six reports, which show three first votes for one side and two for the other, say three for A. A is kept under a *recovery*

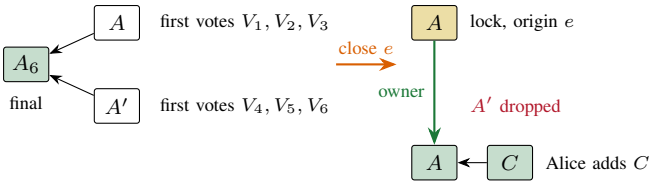


Fig. 2. A double spend at position 7. Neither block reaches four first votes. The five selected reports show three votes for A and two for A': the checkpoint locks A for recovery and drops A', which can never become final. Alice recovers by signing one new block C on top of A; when C is final, so is A.

lock of origin e and A' is dropped: from now on no validator admits A' or any other rival of A at position 7, so A' can never become final. Should the five reports show three against three, which needs a reporter who voted both sides, nothing is kept at position 7.

How Alice recovers. RAI guarantees that Alice gets her account back once she stops contradicting herself (Theorem 3). She reads the decided checkpoint from any validator and acts on what it says:

- If it kept A, she signs one new block C at position 8 on top of A; C may carry a payment or nothing at all. Because A is kept only for recovery, C is admitted once A has four first votes again at position 7, which only A can now collect. Nothing conflicts with C, so it then collects four first and four final votes and becomes final, and finalizing C finalizes A below it (Figure 2).
- If it kept nothing at position 7, she publishes one block there, A, A', or a new one, and only that one; it is uncontested and becomes final like any other block.
- If one side is already final, there is nothing to recover.

She cannot pick the side the checkpoint dropped: a block on top of A' is refused while the lock on A stands. In the normal case recovery takes until the checkpoint after the fork is installed, plus two confirmations.

Alice often does not need to do anything: in the next epoch the validators vote A again, those that voted A' included, since the decided checkpoint proves that no epoch-e certificate for A' can form, and A becomes final on its own. This is a convenience, not a guarantee: a validator may already have spent its one first vote of the new epoch. For any forked position RAI guarantees that it is kept identically on every validator, that at most one side ever becomes final, and that it blocks nothing outside the account. Recovery itself needs a lock that the dropped side cannot release, which holds unless a Byzantine validator supported both sides (Theorem 3).

Durable votes. A correct validator writes its first vote and its final vote to disk before releasing them, so that a crash and restart cannot make it vote twice. Our prototype syncs these writes; Section X reports what that costs.

VI. Closing an Epoch

An epoch closes in three steps: every outgoing validator freezes a report, the outgoing committee agrees on a check-

report V_1	A_6: F	A: N	B: voted
report V_2	A_6: F	A: N	B: voted
report V_3	A_6: F	A: N	B': voted
report V_4	A_6: F	A: voted	B: voted
report V_6	A_6: F	A': N	

checkpoint S_e: A_6 final (inherited); A *locked, notarized* (V_1, V_2, V_3 show it notarized; V_6's claim for A' has no four first votes behind it and is dropped);
B *locked for recovery, origin* e (three reporter first votes, no rival with three); B' dropped.
Promotion: A is the only block kept at position 7 and has an epoch-e notarization $\Rightarrow$ A final in S_e.

Fig. 3. Five of six reports close epoch e for Alice's account. F = final, N = notarized, voted = first-voted but not notarized (the report's second set).

point built from $N-f$ reports, and the next committee installs it. Figure 3 follows Alice's account through one close.

Reports (R3). When a validator's local clock says the epoch is over, it stops first- and final-voting in it, and signs one header committing to two sets. The first is a tagged view of its ledger: every block it holds, tagged *final* (by an inherited checkpoint or by a certificate it has seen), *notarized* (four first votes seen, no finality), or *inherited* (kept by the previous checkpoint under a recovery lock, nothing new seen). The second set lists every block the validator first-voted in this epoch that the first set does not already show as notarized or final. The two sets are frozen: later events change the validator's live view, never its report. The header carries only the hashes of the two sets; whoever needs the sets reconstructs them (Section IV). A report is *usable* at a validator once both sets are reconstructed and every claim in them is backed by evidence that validator holds: four first votes behind every "notarized", a certificate behind every "final", the reporter's own signed first vote behind every entry of the second set. A Byzantine reporter can make its own report unusable, and nobody else's.

The second set has a subtle requirement, found by model checking. A reporter that first-voted a block the previous checkpoint already keeps under a recovery lock tags it *inherited* in the first set. If the second set excluded every block of the first set, as our earlier definition did, this fresh vote would appear nowhere, and a latent fast certificate could vanish from the checkpoint. The second set therefore excludes only blocks tagged notarized or final.

The recipe (R4). A candidate checkpoint names exactly $N-f$ usable reports from distinct members, in a fixed order, plus a manifest of the evidence behind their claims. From these inputs every validator computes the same state by the following recipe, which is deterministic and never consults the candidate's own certificate.

1) *Keep finality.* Every block a selected report shows final, with its ancestors and the sends its receives depend on, is final.
2) *Lock notarized blocks.* At each position, if a selected report shows a block notarized, backed by four first votes, keep it under a *notarization lock*. Two conflicting blocks cannot both be notarized (Section V), so there is at most one.
3) *Lock for recovery.* If no block at the position is notarized, but exactly one block has first votes from r (3) of the selected reporters, keep it under a *recovery lock* with origin e. If several conflicting blocks reach r, which needs a reporter who voted both, nothing is locked and the position is voted afresh in the next epoch; this is safe because a block that can still become fast-final has no rival with r supporters.
4) *Carry old locks.* Every lock of the previous checkpoint is carried forward, unless a final block excludes its block, or, for a recovery lock of origin o, the manifest contains an exclusion witness of epoch o for a conflicting block. Nothing else releases a lock; in particular a newer epoch number never does.
5) *Promote.* For each account, starting at its last final block, walk up while the next block is the only one kept at its position and has an epoch-e notarization of first votes; those blocks become final with the checkpoint. Promotion never resolves a fork and never promotes a block kept by a recovery lock alone.

The thresholds make this safe (Theorem 1): whatever the old epoch could still confirm is kept as final or under a lock, and a lock can only be released by evidence of its own origin epoch, which cannot exist if the block it protects can still be final.

Why locks have an origin. A recovery lock says "three validators first-voted this block in epoch o; a fast certificate with five epoch-o votes may exist." Such a certificate is excluded exactly by four epoch-o supporters for a rival, and by nothing else: four epoch-$(o+1)$ votes for the rival say nothing about epoch o. So the lock records o, and a block may carry locks of several origins, one per epoch in which it was voted but not resolved. Releasing a lock on the strength of a newer epoch's votes was our second wrong rule; it let two validators finalize different blocks.

Eligibility, agreement, and installation (R5). A candidate is *eligible* if its $N-f$ reports are usable, its manifest hash matches, and the recipe yields the state whose digest it names. The outgoing committee runs the agreement service with this predicate; the service may be any Byzantine agreement protocol with external validity. The decided value is small, $N-f$ report headers and two digests, whatever the number of accounts. The next committee verifies the decision proof, reconstructs the inputs, recomputes the state, and installs it: its final set is the union of its base and what it had already finalized, replay applies each operation once, and nothing is rolled back. There is no second vote; the successor only checks.

After the checkpoint: what is guaranteed. A lock guarantees that no rival of the kept block is admitted at its position until evidence of the lock's own origin releases it; a notarization

lock is never released. A lock does not by itself make the kept block final. What finalizes it is a block on top: the owner extends the kept block with a fresh child, which may be empty, and since the child has no rival it is final within a window in which q correct validators are voting, finalizing the kept block below it (Theorem 3). A block under a notarization lock may be extended at once, one kept only by a recovery lock only once it is notarized again in the open epoch. Our earlier rule let any kept tip be extended, which was unsafe: a dropped side with an old witness may become final in the new epoch while a child's certificate finalizes the kept side (supplement, Case 5). Where the checkpoint kept nothing, the owner publishes a single block at the position.

After the checkpoint: what usually happens. A kept block also gets an instance at its own position in the open epoch, and validators that voted the dropped rival may vote it now, since the rule against switching (R10) ends when the checkpoint is decided. So the kept block often becomes final without the owner, and the same votes admit the owner's child. Nothing guarantees it in the first epoch after the fork: a validator that already spent its first vote there cannot vote again, and the next checkpoint then keeps the block again with a new origin. In our fork workloads, whose owners never extend a forked position, this route alone resolved most forks (Section X).

VII. Voting While the Previous Epoch Closes

A validator opens epoch $e+1$ the moment it leaves epoch e, typically a few seconds before S_e is decided. During this overlap two things could go wrong: the new epoch could confirm a block that the old epoch's latent votes contradict, and the checkpoint could later rule against something the new epoch already confirmed. Rules R6 to R10 prevent both. We explain them with Alice's account at position 8, right after A became final at position 7.

The gate (R6). The simplest safe behaviour is to wait: a block of epoch $e+1$ collects first and final votes as usual, but no validator treats it as final until it has installed S_e and re-checked the block against it. If Alice's block B at position 8 is published during the overlap, its certificate is held, and the moment S_e arrives the held certificate takes effect, or is discarded if S_e kept a conflicting block. This is correct but it is also the stop-and-drain cost in disguise: every block published during a close waits for the close to end. Our measurements of the gate alone showed confirmations stopping for one to three seconds at every boundary (supplement).

The overlap rule (R7). A block can skip the gate if the old epoch provably cannot contradict it. The proof is an exclusion witness in the closing epoch: four supporters for B in epoch e. If four validators supported B at position 8 in epoch e, no rival at position 8 can be notarized or final in epoch e (Section V), so S_e cannot finalize a rival, cannot lock a rival for notarization, and any recovery lock S_e places on a rival has origin e and is released by exactly this witness. So a validator that holds the epoch-e witness for B, four epoch-$(e+1)$ first votes for B, and an epoch-$(e+1)$ final certificate for B may finalize B before S_e is known. The same check must hold for every unresolved block below B on Alice's chain, because finalizing B also finalizes its ancestors; and if B bypasses a recovery lock inherited from an older checkpoint, the validator must hold a witness of that lock's own origin epoch. The rule is exactly the check a validator would perform after installing S_e, so a validator that finalizes B before and one that re-checks after reach the same verdict (Lemma 8).

Late notarization (R8). A block published after the boundary has no epoch-e votes at all, so the overlap rule would never apply to it and it would wait at the gate. Late notarization fixes this. A validator that has left epoch e but not yet installed S_e, when it first-votes B in epoch $e+1$, also signs an epoch-e *support* for B, unless it supported a different block at position 8 in epoch e. These late supports count toward the exclusion witness and toward nothing else: no notarization, no final vote, no report entry rests on them, since reports are frozen before they are cast. Safety is the same argument as before: one validator supports one block per position in epoch e, by a first vote or by a late support. In our runs 80 to 97% of the blocks published during a close reached their witness this way and finalized before the checkpoint.

Early votes (R9). Here is where the first of our earlier rules failed. Suppose B conflicts with a block B' that S_e keeps under a recovery lock, and V_3, V_4, V_5 first-vote B early, without the epoch-e witness for B. V_1 later installs S_e, obtains the witness from the Byzantine V_6, releases the lock, and first-votes B too: five first votes, a fast certificate that only V_1 can justify. The next checkpoint may select reports without V_1's and keep B', and a later epoch finalizes B' while B's certificate is still valid (supplement). The rule removes the dependency: a first vote names the checkpoint its signer had installed; *early* votes, cast before S_e, count toward notarization and witnesses but never toward a fast certificate, and an early final vote is released only by a validator that holds the overlap witnesses, which its report then carries to the next checkpoint. A block first-voted during a close therefore takes two rounds instead of one.

No switching (R10). A validator that supported B' at position 8 in epoch e does not first-vote a conflicting B in epoch $e+1$ until it has installed S_e. Otherwise it could contribute to an epoch-$(e+1)$ notarization of B while its epoch-e vote still helps B' toward a latent certificate.

After installation. When S_e arrives, each validator re-checks its provisional epoch-$(e+1)$ work against the finalized history, the frontier, and the locks of S_e. Held certificates take effect; work on a block that S_e excluded is discarded, without clearing the validator's recorded first vote. Every position with one kept block gets an instance at that position in the open epoch, and a rival is refused while any lock on the kept block stands. If a kept block does not finalize in epoch $e+1$ either, its first votes are exposed again in the next reports, and the next checkpoint locks it again with origin $e+1$: a block may carry locks of several origins, each released only by a witness of its own

epoch. Its owner can extend it once it is notarized again in an open epoch.

VIII. Why It Is Safe and Why It Terminates

This section gives the arguments in words; the supplement has the formal statements and full proofs, numbered as here. All arguments are relative to the two services: safety uses the integrity of set reconstruction and the agreement, eligibility, and verifiability of agreement; termination additionally uses their termination properties.

One epoch (Lemma 1, account exclusion). Two sets of q supporters share a correct validator, and a correct validator supports one block per position. So at most one block per position has an exclusion witness, and if a block can become final, by q final votes or by $N - p$ settled first votes, no rival has a witness; if it can become fast-final, every rival has fewer than r supporters.

Notarized ancestry (Lemma 2). A child is first-voted only on a parent that is final, under a notarization lock, or notarized in the child's epoch, never on one kept by a recovery lock alone. So two conflicting blocks final in epoch e, even only as ancestors, meet at one position where Lemma 1 or the lock excludes one.

Reports expose votes (Lemma 4). A correct reporter's frozen report tags a block it final-voted as notarized or final, and lists every other block it first-voted in the second set, whatever the block inherited from the previous checkpoint. Late supports are never in a report. This is why the second set must list fresh votes for blocks the predecessor already kept (Fix B in Section IX): the earlier definition removed them, and a fresh vote vanished.

Theorem 1 (Closure preserves latent and late finality): If an epoch-e final certificate for B can be formed from released votes, even one completed after the reports froze or after S_e was decided, then every eligible checkpoint S_e holds B: as final, under a notarization lock, or under a recovery lock of origin e. No admissible evidence releases that hold.

Why. Count overlaps. A normal certificate has q final voters; the checkpoint selects $N - f$ reports; the two share at least $p+1$ correct validators, each of which notarized B before final-voting and says so in its report, so the recipe keeps B final or locks it. A fast certificate has $N-p$ first voters; it shares at least r correct validators with the reports; if none of them tagged B notarized, each lists B in its second set, so B has r reporter votes while every rival has fewer (Lemma 1) and no rival is notarized; the recipe places a recovery lock. If B bypasses an older lock, every correct signer of its certificate holds the releasing witness (Lemma 5), and the manifest carries it. A lock of origin e on B can only be released by an epoch-e witness for a rival, which Lemma 1 excludes. A B final only as an ancestor is held below its descendant, and Lemma 2 leaves no rival a witness to release it. The argument uses only the signers and the selected reporters, so it does not matter when the certificate is assembled.

Lemma 8 (Same verdict before and after installation): A validator that finalizes B under the overlap rule before installing S_e accepts B if and only if it would accept B by re-checking after installing S_e; and a later checkpoint verifies a report that claims B final.

Why. The overlap rule asks for exactly the evidence that re-checking consults: the epoch-e witness for B rules out a final or notarized rival in S_e and releases any origin-e lock on a rival; the matching-origin witnesses release any older lock; and S_{e-1} was already installed when the overlap began. Without matching-origin release (the second earlier rule) two correct validators reached different verdicts on the same evidence depending on arrival order.

Theorem 2 (Transfer safety across epochs): Finalized histories at correct validators are compatible and never rolled back, finalized balances are nonnegative, and each send is received at most once.

Why. By induction over epochs, jointly with Theorem 1, which each step uses for the closure before it. Inside an epoch Lemma 2 brings two conflicting final blocks to one position, where Lemma 1 or a notarization lock excludes them. At a boundary, agreement gives every correct validator the same eligible checkpoint, Theorem 1 says it preserves every formable old certificate, and promotion (R4) only finalizes a block with a closing-epoch notarization, which excludes rivals. For the overlap, the first unresolved position of an early-final prefix has an epoch-e witness, so no closing-epoch certificate or promotion can contradict it, and Lemma 8 makes all correct validators agree on which overlap blocks are final. The checkpoint's construction is deterministic in its inputs (Lemma 9), so correct validators compute the same state, and installing is a union of final sets with duplicate-suppressed replay, never a rollback.

Theorem 3 (Termination): Every started handover is eventually decided and installed by correct validators. An owner whose new blocks are uncontested sees them final and in a checkpoint eventually. If a checkpoint keeps a block at a forked position under a lock that no rival can release, the owner recovers the account by extending that block; a recovery lock is releasable only if a Byzantine validator supported both sides.

Why. A correct reporter's report is finite and backed by evidence it holds, so under set reconstruction and the data service it becomes usable at every correct validator; $N - f$ usable reports make an eligible candidate constructible, and agreement decides one; successors verify the proof and install. For accounts: in a window where $N-f \geq q$ correct validators are open, an uncontested block collects q first votes, hence a notarization and q final votes; a block caught by a boundary is preserved by Theorem 1, an omitted one is retried. At a fork, a recovery lock rests on r epoch-o supporters of the kept block, so a rival's witness of q needs a validator that supported both ($r + q = N + 1$). Without one, from the second epoch after the checkpoint every correct first vote at the position goes to the kept block, and the owner's child follows. With one, a Byzantine validator can keep the position open by rebroadcasting both sides and withholding the vote either needs, as where the checkpoint kept nothing; the position stays

identical everywhere, with at most one side final, blocking nothing outside the account.

IX. Model Checking the Handover

Three of the rules in Sections VI and VII replaced earlier rules that written arguments had accepted; a fourth, re-notarization, was found by review. We wrote five TLA$^+$ models of the handover and checked them with TLC. Each takes one account with $N = 6$, $f = p = 1$, five correct validators and one Byzantine validator whose signatures are available for every block, vote kind, and epoch, and whose report is chosen by the adversary subject to the usability checks. Agreement is the nondeterministic choice of one eligible candidate; TLC explores every interleaving, with validators identified up to permutation. The models, configurations, and TLC outputs with the violating traces are in the supplement and the artifact.

Close three voting epochs and two checkpoints, no early voting; the recipe with locks of several origins and matching-origin release. Switch: the old or the new definition of the report's second set.

Overlap one closing and one open epoch; validators cross their boundary one by one, vote early, late-notarize, and finalize under the overlap rule or after installing. Switches turn off matching-origin release, late supports as evidence, and the separation of notarization from late supports.

Handoff early voting, two checkpoints, and a committee change with the Byzantine validator in both committees; the early-vote rule on or off; a releasing witness reaches the second checkpoint only through a selected correct report, which models a withheld signature.

Prefix two positions of one account: a parent with a rival at position 1 and two children at position 2, so that the prefix-wide guard of R7 is exercised; committees identical, disjoint, or sharing members. Switches check only the finalizing block instead of the prefix, and let a child inherit its parent's witness.

Recovery a child of each forked block at the next position, installation, and settled votes. Switch: the earlier retained-tip exemption.

Table II summarizes. With the old second set a fresh vote for a recovery-locked block appeared in no report, and two conflicting fast certificates were reachable; without matching-origin release two correct validators finalized different blocks depending on whether the checkpoint or the certificate arrived first; with the fast path counting every first vote, Theorem 1 failed at the second checkpoint by the trace of Section VII, with identical and with disjoint committees. In the prefix model, checking only the finalizing block or letting a child inherit its parent's witness finalizes conflicting children once the committees differ; the prefix-wide guard passes every $N = 4$ configuration, and its $N = 6$ runs were still in progress at submission. The recovery model finds the retained-tip exemption unsafe for every pair of committees, even without early votes. Probes show that the invariants are not vacuous.

TABLE II
TLC results. States are distinct states of completed runs; a number of steps is the length of the counterexample.

Model, switches	Result	States
Close, old second set	Thm. 1 fails (19 steps); two fast certificates (23)	–
Close, Fix B	no violation	0.65 M
Overlap, all repairs	no violation	5.2 M
Overlap, no release	same-verdict fails (15)	–
Overlap, late votes not evidence	next close cannot verify a correct report (14)	–
Overlap, late votes in notarization	a correct report becomes unusable (9); safety holds	5.5 M
Handoff, old fast path, same or disjoint K	Thm. 1 fails for S_2 (22, 23)	–
Handoff, rule, same / disjoint / 3 shared	no violation	8.3 / 3.7 / 121 M
Prefix N=4, prefix guard, all overlaps	no violation	0.3–4.0 M
Prefix N=4, target only, 2 shared	conflicting finality (13)	–
Prefix N=4, child inherits witness, not same K	conflicting finality (12–14)	–
Prefix N=6, target only or inherited witness	conflicting finality (simulation)	–
Recovery N=4, tip always extensible, all K	conflicting finality (16)	–
Recovery N=4, re-notarization, all K	no violation	0.7–3.9 M

The models are small: one or two positions, two or three epochs, no receives, no restarts, $N \in \{4, 6\}$. Their abstractions only add behaviours, so a violation in the model is a violation of the protocol, but a clean run is evidence, not proof.

X. Implementation and Evaluation

Prototype. We implemented RAI in RsNano [9], a Rust implementation of the Nano node [5]. Account elections follow Section V with the thresholds of Table I ($f = p = 1$, $N = 6$), late notarization, the early-vote rule, and the prefix-wide overlap rule; checkpoints carry origin-stamped locks and are built by the recipe of Section VI. Epochs end at fixed times $T_0 + kD$ on every validator's local clock. Every first, final, late, and close-election vote, frozen report, decided checkpoint, and overlap certificate is written to its own LMDB environment and synced before a signature that depends on it leaves the process; a restarted validator reloads these records, refuses whatever they forbid, and fetches the checkpoints it missed. Set reconstruction uses rateless invertible Bloom lookup tables [7]: a validator that needs a report starts from its own view of the closing epoch and asks any holder for coded symbols until the symmetric difference peels, then checks the signed root. Agreement is one leader-based election per handover in the style of Kudzu [8]: a validator votes for a

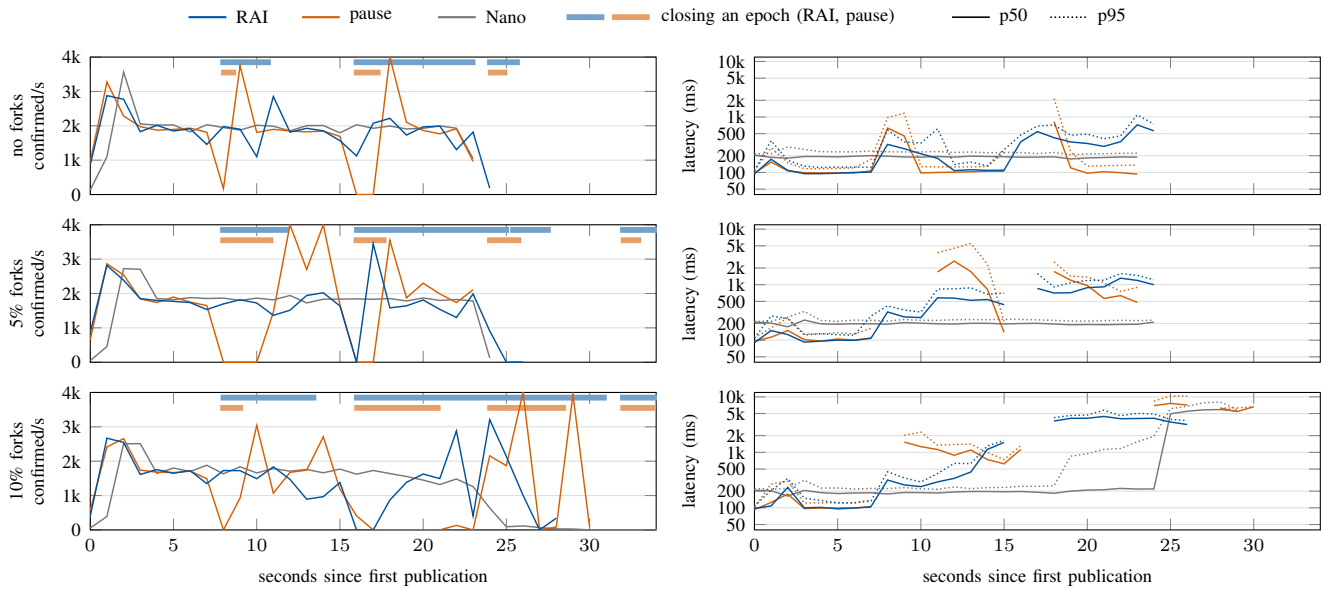


Fig. 4. Throughput and latency over time at an offered 2,000 blocks/s on loopback. Left: non-fork blocks confirmed per second (clipped at 4,000). Right: median (solid) and 95th percentile (dotted) latency of the blocks confirmed in each second. One run per system and variant, the one with the median 95th percentile of three. The bars at the top of the left panels mark when RAI and pause are closing an epoch, from the first validator leaving it to the last one seeing its checkpoint decided. Pause stops confirming during its closes and then catches up in a burst; RAI keeps confirming through longer closes, at a higher latency.

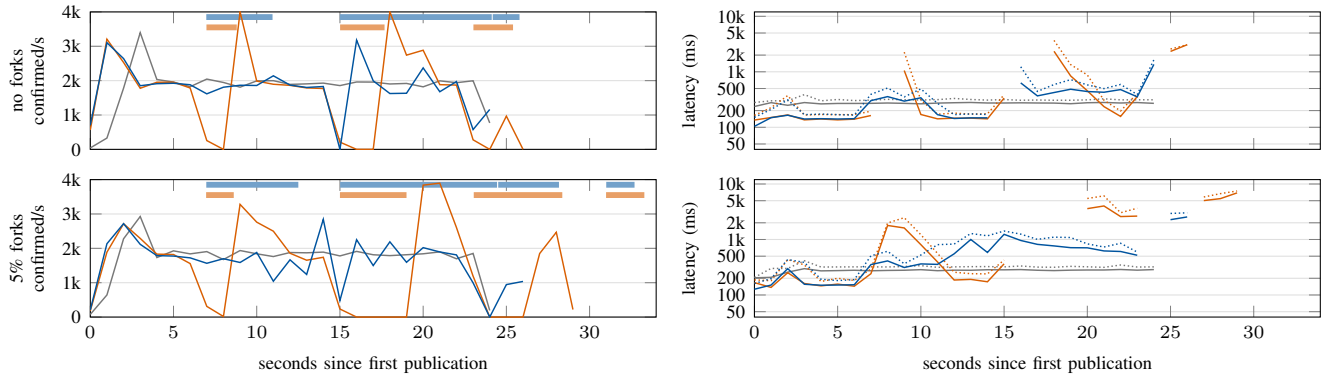


Fig. 5. As Figure 4, with an emulated one-way delay of 40 ms on every message every validator sends.

proposal only after reconstructing its reports, checking the manifest, running the recipe, and matching the state digest, which gives eligibility; leaders rotate, and a round whose leader is silent times out after 2 s. The supplement lists the prototype's departures from the protocol and the defects the measurements found.

What we compare. Three systems run the same workload. *RAI* is the protocol of this paper. *Pause* is RAI with one switch flipped: no instance of epoch $e+1$ starts before S_e is decided locally, so the overlap rule and late notarization never apply. It is the stop-and-drain transition that RAI set out to avoid, with everything else (reports, recipe, agreement, installation) identical. *Nano* is the unmodified RsNano `develop` node (commit `ff2b32710`), which runs Nano's open representative voting: a block is confirmed once final votes reach 67% of the online weight, after a round of non-final votes; it has no epochs, no checkpoints, and no committee, and since final votes cannot be withdrawn, a fork whose votes split leaves neither side able to confirm. All three are driven by the same load generator, built from the RAI branch.

Setup. All validators run as separate processes on one MacBook Air (Apple M1, eight cores, 16 GiB) over loopback, so there is no network delay and CPU contention sets the delays; a second series adds an emulated one-way delay of 40 ms to every message every validator sends. Six equal-weight representatives hold the voting weight. A load generator funds 45,000 accounts and then publishes 45,000 blocks at an offered 2,000 blocks/s, each on the frontier of a randomly chosen account: it builds on a confirmed frontier, or extends an unconfirmed non-fork frontier with a fresh child once it has waited

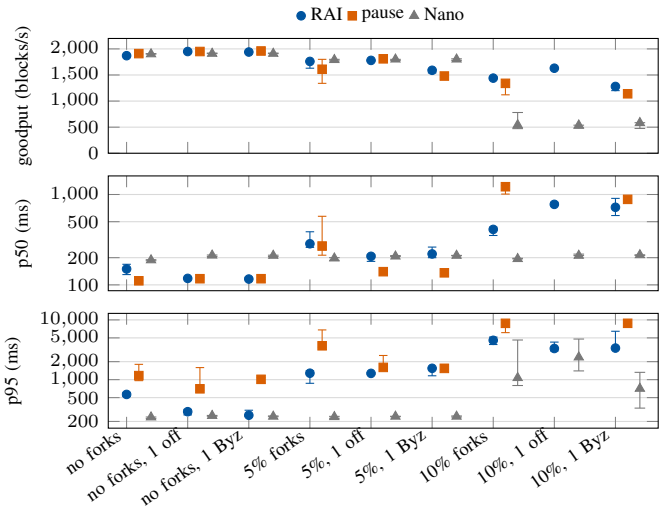


Fig. 6. All nine variants on loopback, three runs each: goodput over the whole run, median and 95th-percentile latency; marks are the median run, bars the spread. A missing mark is a variant whose client never saw every non-fork block confirmed in any of its runs. Averaged goodput hides the stops that Figure 4 shows, because at this offered load each system catches up after them.

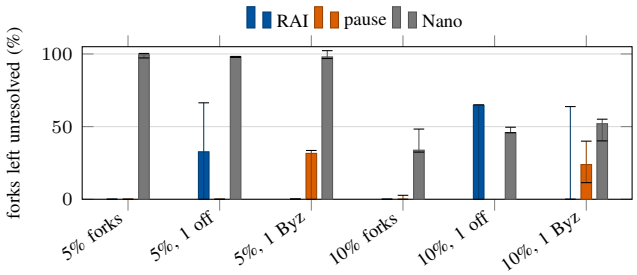


Fig. 7. Forked positions with neither side final when the run settled, as a share of the forks published (about 2,250 at 5%, 4,500 at 10%); median of three runs with spread. The generator never builds on a forked position, so this measures what validators resolve without the owner, a route RAI does not guarantee (Section V).

5 s, and it never builds on a position where it published a fork. With forks enabled it publishes a conflicting sibling for 5% or 10% of the blocks, each sibling to half of the validators. An *offline* validator runs no node but keeps its weight; a *Byzantine* validator also runs no node, and the generator votes with its key at random, choosing any vote kind for recent blocks and contradicting itself. RAI and pause use 8 s epochs and sync every signing record before releasing it. We report non-fork blocks only: *goodput* is the number of non-fork blocks confirmed divided by the time from first publication to last confirmation, and *latency* runs from a block's first publication to its confirmation at one validator. A run *settles* when every running validator holds the same cemented state and, with epochs, has decided the same checkpoints. Every variant ran three times per system, interleaved, and the figures show the median run with the spread of the three. The run records list three runs we excluded because the host slept during them or an operator error killed them, and their replacements.

Without forks. All three systems keep up with the offered load, at 1,870 to 1,960 non-fork blocks/s (Figure 6). RAI's median latency, 116 to 150 ms, is below Nano's 188 to 212 ms (Figure 6): an uncontested block finalizes on one round of five settled first votes, where Nano always needs a round of final votes after its quorum of non-final votes. RAI's tail is longer than Nano's (95th percentile 254 to 566 ms against 235 to 249 ms); that is the cost of having checkpoints at all, since a block that lacks one of the four late supports waits for the checkpoint, and a close takes 4 to 7 s. In one RAI run the client never saw one block confirmed although every validator had cemented it, so that run has no client numbers. The pause transition, with the same checkpoints, has a median like RAI's but a 95th percentile two to four times longer (703 to 1,168 ms): every block published during a close waits for it.

Over time. Figure 4 shows why, second by second, for one run per system and fork rate. Under pause, confirmations stop for two to three seconds at every boundary, and up to six at 10% forks, until the checkpoint is decided, and the blocks confirmed right after carry the length of the close in their latency. Under RAI, confirmations continue through the close; they dip for one to two seconds at a boundary, while each validator freezes its report and carries thousands of open instances into the new epoch. Nano has no boundaries and confirms steadily.

With forks. RAI and pause settled every fork run, with every running validator deciding the same checkpoints and cementing the same blocks, except one pause run at 10% forks whose validators still disagreed on 347 blocks when the settle wait ended. Under pause, the client also never saw some non-fork blocks confirmed (0.6 to 4% of them) in all three 10%-fork runs with a validator offline and in two of three with a Byzantine validator; those runs have no client numbers. Nano settled every run except two of the three without faults at 10% forks, whose validators still disagreed by a few blocks when the settle wait ended. Latency grows with the fork rate for both epoch-based systems, because forks make reports, manifests, and closes larger: RAI's median is 207 to 285 ms and its 95th percentile 1.3 to 1.5 s at 5% forks, against 136 to 269 ms and 1.5 to 3.7 s for pause; at 10% forks RAI's 95th percentile is 3.3 to 4.6 s against 8.8 s for pause. Nano's latency is unaffected at 5% forks (95th percentile 238 to 242 ms), but at 10% its goodput falls to 536 to 582 blocks/s, a third of RAI's 1,280 to 1,630; we did not analyse why.

Forks left unresolved. Figure 7 shows how many forked positions had neither side final at the end. Because the generator never builds on a forked position, this measures what the validators resolve without the owner, which RAI does not guarantee (Section V). Nano resolved almost none of its 5% forks (about 99% left open) and half of its 10% forks. RAI and pause resolved every fork in most runs. RAI left 1,491 of about 2,250 forks open in one of three 5%-fork runs with a validator offline, about 2,900 of about 4,500 in two of three

10%-fork runs with a validator offline, and 2,885 in one 10%-fork run with a Byzantine validator. In each of these, two causes in the prototype combine. With a validator offline, a position split three against two looks to each validator as if it could still be notarized, so at the boundary every validator re-votes its old side and the split repeats in the next epoch; with all six online the split is recognised and the kept block is voted by everyone once the checkpoint is decided. And once the load stops, no new election starts an epoch's timer, so no further checkpoint is decided. The open positions are exactly those the last checkpoint kept, identically on every validator.

Link delay (Figure 5). With 40 ms added to every message, every close takes longer, and RAI's median rises to 251 to 303 ms without forks and 431 to 487 ms at 5% forks. The rise comes from the closes, not from ordinary confirmations: outside a close RAI's median is 140 to 148 ms, but its closes now take 4 to 9 s of every 8 s epoch, and the half of the blocks confirmed during a close need about 430 ms, which is consistent with the second round that early votes require; we did not break latency down by finalization path. Pause keeps a lower median without forks (168 to 183 ms) but its 95th percentile grows to 2.4 to 2.8 s, three times RAI's 740 to 858 ms; at 5% forks pause is worse on both (median 689 to 984 ms, 95th percentile 4.6 to 5.7 s against 1.3 to 1.5 s). Nano, which always takes two rounds, rises to a median of 272 to 274 ms and a 95th percentile of 315 to 323 ms without forks, and 277 to 285 ms and 322 to 328 ms at 5% forks: RAI's median now matches Nano's, and outside closes it is about half of Nano's (140 to 148 ms against 273 ms), while its tail stays two to four times longer. The per-second view (Figure 5) shows the same pattern as on loopback: pause stops at every close, RAI keeps confirming at a higher latency during closes that take 4 to 9 s, and Nano is flat.

Restarts, committee changes, and handoff. A second harness kills validators mid-load or as a close starts, restarts them on their data, and checks every logged signature for conflicts. Restarted validators rejoined by checkpoint catch-up. With eight validators of which six form the committee, replacing two members, replacing them with a Byzantine member in every committee, and replacing every correct member of a four-member committee all settled with identical checkpoints everywhere, and a departing member killed when its last close starts and kept down 30 s left every checkpoint decided identically. No run produced a conflicting signature. Synced signing costs about 130 ms at the median and nothing in goodput. The supplement reports these runs in detail.

Threats to validity. All validators share one host, so delays are CPU-bound even with emulated link delay; three runs per variant make the spreads indicative, not statistical; the Byzantine validator votes at random rather than following a script; and process kills test crash consistency, not power loss.

XI. Related Work

Guerraoui et al. characterize the consensus number of asset transfer [1]. FastPay settles prefunded payments without a global order [2]; Astro builds asynchronous payments on broadcast and, in its extended version, discusses replica reconfiguration with view-tagged messages and account-log transfer [3], [10]; Zef adds private coins [4]; Nano uses account chains with separate send and receive blocks and open representative voting [5]. RAI's difference is the frozen-report evidence that preserves first- and final-vote obligations across a boundary while the next epoch is already voting, and the locks that give a contested position a decision, not a deadlock.

Sui Lutris uses consensus for shared objects and checkpoint sequencing, including end-of-epoch handling of processed certificates [11]; it sequences certificates continuously, so it has a stream to drain, where RAI has none. Cuttlefish broadens the fast path with collective objects and FastUnlock [12]; RAI's recovery of a locked account is narrower and needs no new primitive. Pastro studies permissionless asynchronous asset transfer under a dynamic adversary [13]; dynamic Byzantine reliable broadcast and asynchronous Byzantine reconfiguration address broader changing-participation settings [10], [14]. PDCC combines certificate-based payments with membership change and periodic settlement; its installation rule accepts certificates for the installed configuration and assumes correct members common to consecutive configurations [15], whereas RAI preserves old-epoch certificates assembled later and needs no members in common.

Agreement with external validity is a standard interface [6]; we instantiate it with a leader-based election in the style of Simplex and Kudzu [16], [8]. Authenticated set reconciliation is mature [17], [18], [7]. Model checking small configurations of Byzantine protocols with TLC is common practice [19]; here it served as a regression test for the handover rules.

XII. Limitations and Conclusion

RAI's checkpoints grow with the unresolved forks they keep, and locks, late supports, and overlap witnesses must be kept while a lock they can release survives; we give no garbage-collection policy beyond the prototype's four-epoch cap. The committee model is authenticated and equal-weight with an externally kept fault bound. The evaluation runs on one host, so the delays it measures are dominated by CPU contention even with emulated link delay, and process kills test crash consistency, not power loss. The models cover at most two positions and three epochs.

Within these limits RAI does what it set out to do. It keeps account-local payments safe across committee changes without stopping: the next epoch votes while the previous one closes, immutable reports preserve whatever the old epoch could still confirm, and four overlap rules keep the two epochs from contradicting each other. It keeps at most one side of every contested position, identically everywhere, and lets the owner recover the account unless a Byzantine validator backed both sides. Against a stop-and-drain transition built from the same code, it keeps confirming through every boundary and shortens the latency tail up to fourfold; against a system without epochs it pays in latency tails, not in throughput, and in return it can change its validators.

AI Tool Usage

ChatGPT assisted with an earlier restructuring of the manuscript, with source checking, protocol review, and proof exposition, and helped write the TLA$^+$ models, which were checked with TLC. Claude Code (Anthropic) assisted with the prototype implementation, the benchmark harness and the stop-and-drain and link-delay switches, ran the experiments of Section X and produced their figures, diagnosed the defects listed in the supplement, and drafted the text and figures of this revision, which the authors revised. The proof arguments were reviewed by the authors and have not been machine checked beyond the finite configurations of Section IX. No data in this paper is AI-generated: all measurements come from the prototype runs described in Section X.

References

[1] R. Guerraoui, P. Kuznetsov, M. Monti, M. Pavlovic, and D.-A. Seredinschi, "The consensus number of a cryptocurrency," in *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*, 2019, pp. 307–316.

[2] M. Baudet, G. Danezis, and A. Sonnino, "FastPay: High-performance byzantine fault tolerant settlement," in *Proceedings of the ACM Conference on Advances in Financial Technologies (AFT)*, 2020.

[3] D. Collins, R. Guerraoui, J. Komatovic, M. Monti, A. Xygkis, M. Pavlovic, P. Kuznetsov, Y.-A. Pignolet, D.-A. Seredinschi, and A. Tonkikh, "Online payments by merely broadcasting messages (extended version)," 2020, extended version of the DSN 2020 paper; Appendix A describes asynchronous reconfiguration.

[4] M. Baudet, A. Sonnino, M. Kelkar, and G. Danezis, "Zef: Low-latency, scalable, private payments," in *Proceedings of the Workshop on Privacy in the Electronic Society (WPES)*, 2023, pp. 1–16.

[5] C. LeMahieu, "Nano: A feeless distributed cryptocurrency network," White paper. [Online]. Available: https://content.nano.org/whitepaper/Nano_Whitepaper_en.pdf

[6] C. Cachin, K. Kursawe, F. Petzold, and V. Shoup, "Secure and efficient asynchronous broadcast protocols," in *Advances in Cryptology – CRYPTO 2001*, ser. LNCS, vol. 2139. Springer, 2001, pp. 524–541.

[7] L. Yang, Y. Gilad, and M. Alizadeh, "Practical rateless set reconciliation," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 595–612.

[8] V. Shoup, J. Sliwinski, and Y. Vonlanthen, "Kudzu: Fast and simple high-throughput BFT," in *39th International Symposium on Distributed Computing (DISC 2025)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 356. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, pp. 42:1–42:19.

[9] RsNano contributors, "Rsnano software repository," Software repository, base of the RAI prototype and of the evaluated baseline. [Online]. Available: https://github.com/rsnano-node/rsnano-node

[10] R. Guerraoui, J. Komatovic, P. Kuznetsov, Y.-A. Pignolet, D.-A. Seredinschi, and A. Tonkikh, "Dynamic byzantine reliable broadcast," 2020, extended technical report, version 2.

[11] S. Blackshear, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, X. Li, M. Logan, A. Menon, T. Nowacki, A. Sonnino, B. Williams, and L. Zhang, "Sui lutris: A blockchain combining broadcast and consensus," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2024, pp. 2606–2620.

[12] L. Kokoris-Kogias, A. Sonnino, and G. Danezis, "Cuttlefish: Expressive fast path blockchains with FastUnlock," 2023, version 1.

[13] P. Kuznetsov, Y.-A. Pignolet, P. Ponomarev, and A. Tonkikh, "Permissionless and asynchronous asset transfer," in *35th International Symposium on Distributed Computing (DISC 2021)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 209. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, pp. 28:1–28:19.

[14] P. Kuznetsov and A. Tonkikh, "Asynchronous reconfiguration with byzantine failures," *Distributed Computing*, vol. 35, pp. 477–502, 2022.

[15] L. Shangguan, "Transaction-driven dynamic reconfiguration for certificate-based payment systems," 2026, initial preprint, version 1, January 14, 2026.

[16] V. Shoup, "Sing a song of simplex," in *38th International Symposium on Distributed Computing (DISC 2024)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 319. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, pp. 37:1–37:22.

[17] M. T. Goodrich and M. Mitzenmacher, "Invertible bloom lookup tables," in *49th Annual Allerton Conference on Communication, Control, and Computing*, 2011, pp. 792–799.

[18] D. Eppstein, M. T. Goodrich, F. Uyeda, and G. Varghese, "What's the difference? efficient set reconciliation without prior context," in *Proceedings of the ACM SIGCOMM 2011 Conference*, 2011, pp. 218–229.

[19] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.